

real-tr-p8-semanticgloss

An AgentMPI run analysis

Abstract

This is the `--glossary-op semantic` ablation of the parallel book translation experiment: eight real Cursor subagents translate one book, and the glossary `allreduce` that makes their terminology agree is performed by a model-implemented operator (`GLOSSARY_MERGE`) instead of the exact, associative, idempotent `UNION` used by `real-tr-p8-full`. The measured price of that substitution is exactly $p - 1 = 7$ extra agent calls, one per internal node of the binomial reduction tree, carrying 5,441 prompt and 4,873 completion tokens and costing \$0.0894 — 95% of the \$0.0939 total cost difference against the exact run. The measured benefit is smaller than the harness’s own documentation predicts and different in kind. Replaying `UNION` over this run’s own eight term sheets, recovered from the blob store, yields a glossary with the *identical* 203 keys and only 7 differing values (3.4%): the semantic fold lost nothing, invented nothing, and never proposed a rendering that no rank had proposed. Two of those seven differences are the one thing an exact operator provably cannot do — having chosen one transliteration of *Adler*, the operator propagated it to *Miss Adler* and *Miss Irene Adler*, keys `UNION` treats as unrelated. The predicted population divergence did not occur and could not have: under `reduce_bcast` the root’s answer is broadcast by handle, and the trace shows all seven broadcast messages carrying one digest, `c908d2c6e40f`, so all eight ranks hold a byte-identical glossary. The headline $6.9\times$ wall-time inflation over the exact run is *not* the operator’s doing: 742s of this run’s 1,152.9s — 64% — is wall clock during which no rank anywhere was executing anything, waiting for worker processes to attach, against 2.3s of such time in the exact run. The takeaway is that at $p = 8$ on this workload a semantic reduction is a linear-cost, logarithmic-latency way to buy cross-key consistency that `UNION` cannot express, and that its advertised dangers — information loss compounding with depth, and a population believing p different glossaries — were both absent here for reasons that are legible in the trace and that will not survive a change of algorithm or of scale.

1 What this run is

property	value
run	real-tr-p8-semanticgloss
experiment	translation
campaign label	real-tr
declared world size	8
ranks appearing in the log	8
executors	broker $\times$ 8, worker $\times$ 56
events	398
wall time	1,152.90s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	1.21×	total busy time over wall time
parallel efficiency	15.2%	achieved parallelism per rank
idle fraction	84.8%	rank-seconds paid for and unused
peak ranks busy	8	of 8 in the world
serial fraction of active time	46.3%	time with exactly one rank busy
load imbalance	1.35×	slowest rank’s busy time over the mean
coordination share	258.7%	wall time inside collectives
rank reattachments	56	fresh agent processes that took over a rank role
agent calls	31	invocations that reached a model
agent latency p50 / p95	59.52 / 388.12 s	per invocation
messages	56	48 eager, 8 rendezvous
tokens sent	31,906	8,893 deferred under rendezvous
tokens in / out	48,530 / 13,878	prompt and completion
cost	\$0.3538	0.0255 \$/kTok out

3 What the analysis notices

- **[warning]** `halo_exchange/?` reports 0 messages but the fabric logged 14: the collective’s own accounting disagrees with the traffic it produced.
- **[note]** Parallel efficiency is 15.2%: 84.8% of the rank-seconds paid for went unused.
- **[note]** Collectives account for 258.7% of wall time, so this run spent more time coordinating than working.
- **[ok]** 56 rank reattachment(s) occurred, up to incarnation 9: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- **[ok]** All 9 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	235.57	20.4	6	6	10	10	11,998	0.0	finalized
1	136.25	11.8	3	3	5	5	794	0.0	finalized
2	166.53	14.4	4	4	8	8	3,888	0.0	finalized
3	151.68	13.2	3	3	5	5	641	0.0	finalized
4	232.17	20.1	5	5	11	11	9,022	0.0	finalized
5	166.18	14.4	3	3	5	5	795	0.0	finalized
6	171.12	14.8	4	4	8	8	4,104	0.0	finalized
7	138.91	12.0	3	3	4	4	664	0.0	finalized

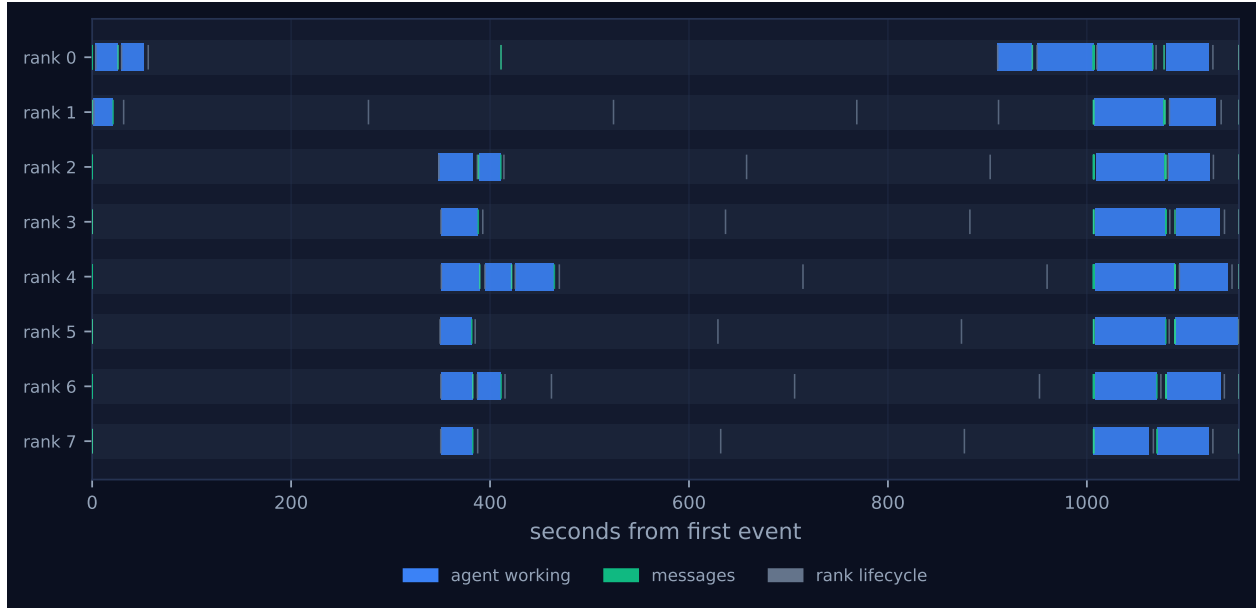


Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

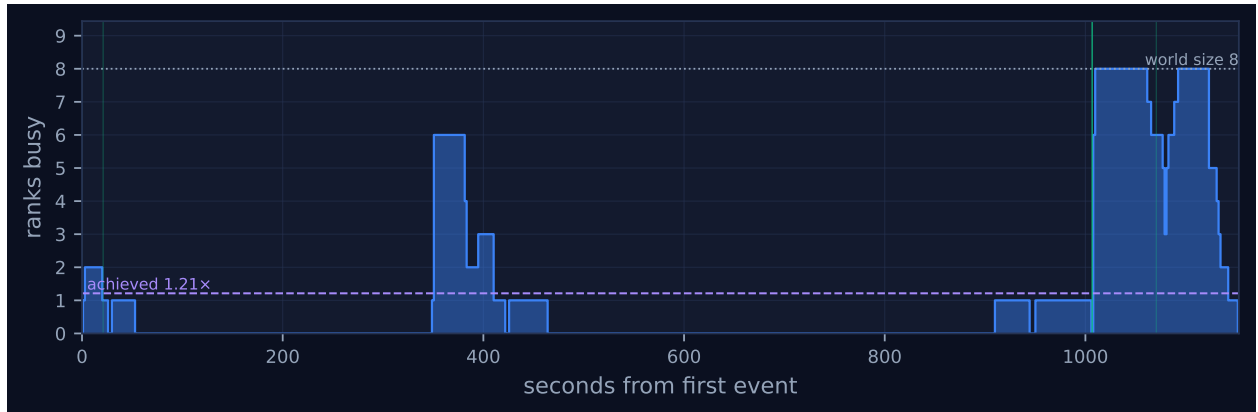


Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

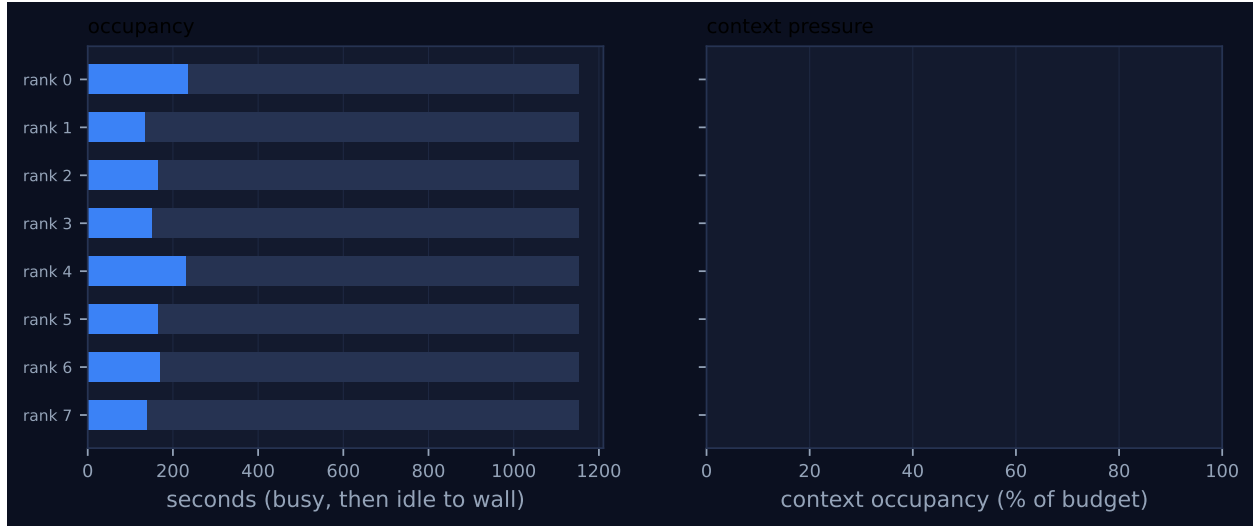


Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	mod
scatter	binomial	decompose	8	8	3	3	7	7	11,594	0.02	0.01	✓
reduce	binomial	glossary-semantic	8	8	3	3	7	7	2,332	980.70	985.68	✓
bcast	binomial	glossary-semantic	8	8	3	3	7	7	10,780	985.70	0.10	✓
allreduce	reduce_bcast	glossary-semantic	8	8	6	6	14	14	0	985.70	0.10	✓
reduce	binomial	register	8	8	3	3	7	7	281	0.13	0.11	✓
bcast	binomial	register	8	8	3	3	7	7	1,295	0.13	0.03	✓
allreduce	reduce_bcast	register	8	8	6	6	14	14	0	0.13	0.03	✓
halo_exchange	?	seams	8	8	0	–	14	–	0	0.00	18.00	–
barrier	central	pre-gather	8	8	0	2	0	0	0	30.20	0.23	✓
gather	binomial	collect	8	8	3	3	7	7	5,059	0.06	0.27	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 does not look like a parallel run. Eight lanes are drawn and the blue bars in them occupy three narrow islands separated by two long stretches in which nothing at all is happening. Figure 2 makes the same point as a step function: one or two ranks busy from 1.7 to 52.4s, six ranks busy for

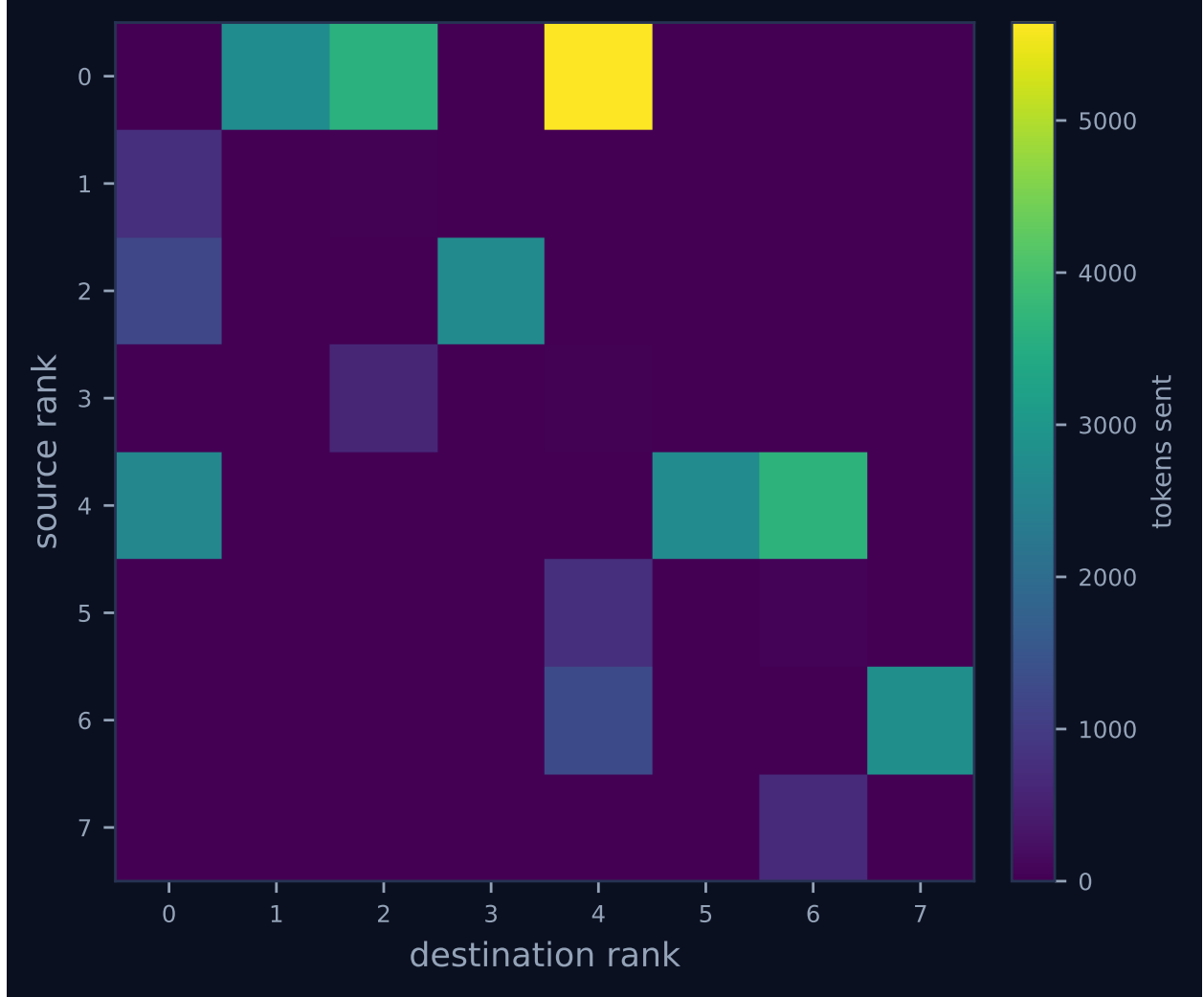


Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

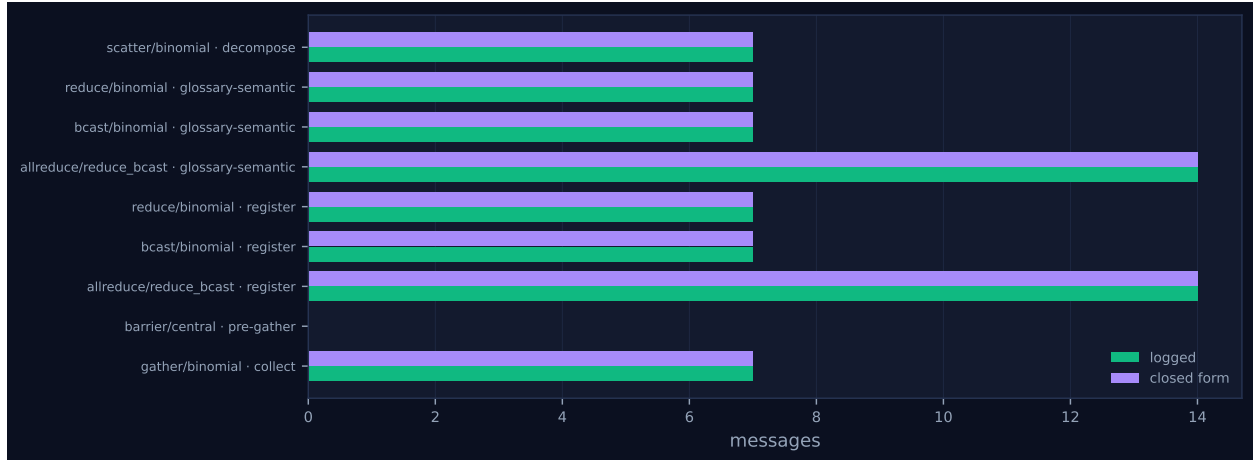


Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

roughly forty seconds around 350–390 s decaying to one by 464 s, a single rank from 910 to 1,006 s, and only then the full eight from 1,007 to 1,152 s. Taking the union of every claim-to-complete interval in the log gives 395.6 s of wall clock with at least one rank executing, so 757 s of the 1,152.9 s span had no work in flight anywhere in the world. Two windows account for 742.3 s of that: 52.4 s–348.5 s (296.2 s) and 464.2 s–910.3 s (446.1 s).

Those two windows are the run’s dominant feature and they are not the semantic operator. The broker log separates queueing from service exactly: every agent call has a `broker.enqueue`, a `broker.claim`, and a `broker.complete`. Summing over all 31 calls gives 2,653.6 s of queue wait against 1,398.4 s of service, so 65.5% of total agent-call elapsed time was spent waiting for a worker to pick the task up. In `real-tr-p8-full` the same sum is 39.2 s of queue against 1,033.2 s of service, 3.7%. Two stalls dominate. Six of the eight `terms:u*` calls were enqueued within 30 ms of the scatter and not claimed until $t \approx 348$ –351 s, because ranks 2 through 7 had no worker attached before then — the first `rank.init` with `executor: worker` for those ranks is at 348.5–351.0 s, whereas in `real-tr-p8-full` all eight workers attached within 1.6 s. The second stall is `reduce:GLOSSARY_MERGE:d2` on rank 0, enqueued at 411.0 s and not claimed until 910.3 s: 499.3 s of pure queueing on the critical path, in which rank 0 held the partially folded glossary and every other rank was blocked in the collective behind it. Rank 0’s incarnation counter tells the story — incarnation 4 attached at 56.4 s and never claimed anything, and incarnation 5 did not appear until 910.3 s.

This has a direct consequence for how the viewer’s statistics row should be read. It reports agent latency p50/p95 of 59.5/388.1 s and `metrics.json` gives a maximum of 534.17 s, which is the `reduce:GLOSSARY_MERGE:d2` call. Only 34.8 s of that 534.17 was model service; the remaining 499.3 s was queue. The `agent.call` event’s `latency_s` field is measured from the harness’s point of view — submission to result — so on a run where workers are scarce it is a scheduling metric wearing a model-latency label. The calibration block inherits the problem: $\alpha_{p99} = 389.6$ s and $\beta = 0.0761$ s/token are fitted to these inflated latencies, and the α/β crossover of 782 tokens should not be believed for this run.

The second island, 348–464 s, is where the reduction tree is visible. Six ranks finish their term sheets almost simultaneously, and then the binomial fold appears as a staircase: rank 2 and rank 6 each perform a depth-1 merge (410.9 s and 411.2 s), rank 4 performs its depth-2 merge (464.7 s), and the

tree narrows to rank 0. Rank 4’s lane in Figure 1 is visibly the widest bar of that island because it did three consecutive agent calls — its own term sheet, its depth-1 fold, and its depth-2 fold. The fan-in shape is also legible in Figure 4: traffic is confined to the binomial edges $0 \rightarrow 1$, $0 \rightarrow 2$, $0 \rightarrow 4$, $2 \rightarrow 3$, $4 \rightarrow 5$, $4 \rightarrow 6$, $6 \rightarrow 7$ and their reverses, with the brightest cell $0 \rightarrow 4$ at 5,644 tokens, most of it the 1,540-token merged glossary travelling down the broadcast tree.

The third island is the run behaving as designed. From 1,006.9 s all eight ranks translate concurrently, then exchange seam boundaries and revise; Figure 2 sits at 8 for most of that stretch, and the dip in the middle is the gap between the last translation completing and the last halo message arriving. The whole productive part of the harness — translate, halo exchange, revise, barrier, gather — takes 146 s and looks exactly like the corresponding phase of the exact run.

One asymmetry in the collective table repays attention. The reduce reports `wall_s` of 980.70 s and the broadcast 985.70 s, which is not two long operations but one: a leaf rank leaves the reduce in 2 ms, having only sent, and then blocks inside the broadcast until the root’s answer arrives, so the leaves’ waiting is charged to the `bcast` and the root’s folding to the `reduce`. Ranks 1, 5, 7 and 3 record `reduce` walls of 1.4–2.2 ms and `bcast` walls of 985.7, 625.2, 624.2 and 618.7 s respectively. Summing those per-rank walls is what produces the 258.7% coordination share: eight ranks concurrently blocked in the same 986 s collective contribute eight times to a total that is then divided by one wall time.

7 Interpretation

7.1 What is true by construction

Three of this run’s most striking numbers were fixed the moment the configuration was chosen. The seven extra agent calls relative to `real-tr-p8-full` are not an empirical finding: a binary reduction over $p = 8$ has $p - 1 = 7$ internal nodes, and `semantic_op` builds a kernel that invokes an agent once per node, so the count is forced. The log confirms the correspondence exactly — the seven calls whose `kind_label` begins `reduce:GLOSSARY_MERGE` are the only agent calls this run has that `real-tr-p8-full` does not, and the other 24 (`terms`, `translate`, `revise`, eight of each) match one for one. The maximum fold depth of 3 is $\lceil \log_2 8 \rceil$, and the labels record the tree level directly: four calls at `:d1`, two at `:d2`, one at `:d3`. And the byte-identity of the result at every rank follows from the algorithm rather than from the operator: `reduce_bcast` computes one distinguished answer at the root and ships it, so `allreduce` correctly records `divergence_risk: false` on all eight of its events even though the underlying `coll.reduce` events all carry `lossy: true`. The two flags mean different things and the trace keeps them apart properly. The per-rank statistics agree: all eight ranks report `glossary_digest c908d2c6e40f`, and the seven broadcast `msg.send` events carry that single digest.

7.2 What the fold actually did

The interesting result is what *emerged*, and it is recoverable in full because every accumulator that crossed a wire was stored in the blob store. The four leaf contributions travelled as `msg.send` payloads with an `acc` field, the four remaining leaf term sheets and all seven fold outputs are stored as blobs, and matching them by key count and containment reconstructs the entire tree. Table 1 is that reconstruction; the service times come from `broker.claim` to `broker.complete` and the output token counts from the corresponding `agent.call` events.

node	depth	$ L $	$ R $	$ L \cup R $	$ \text{out} $	conflicts	unforced	service (s)	out tok.
rank 0	1	38	27	62	62	0	0	22.8	490
rank 2	1	31	25	50	50	1	2	21.2	362
rank 4	1	43	24	59	59	0	0	26.4	456
rank 6	1	31	29	55	55	1	0	23.7	403
rank 0	2	62	50	106	106	2	0	34.8	809
rank 4	2	59	55	108	108	1	3	39.0	813
rank 0	3	106	108	203	203	1	0	56.6	1,540

Table 1: The seven applications of `GLOSSARY_MERGE`, reconstructed from the blob store. “conflicts” counts keys present in both inputs with disagreeing values; “unforced” counts keys the operator rewrote although its two inputs did not disagree about them.

The column that matters is $|\text{out}|$ against $|L \cup R|$. They are equal at every node. Across all seven folds, at depths 1, 2 and 3, the operator dropped zero keys and invented zero keys, and the 203 terms in the final glossary are exactly the 203 distinct source terms the eight ranks proposed between them. For an operator the runtime classifies as lossy (`Associativity.APPROX`, hence `Op.lossy` true, hence `lossy: true` on every `coll.reduce` event), the measured key-level loss on this workload was nil. The declaration is a worst-case contract, not a measurement, and this run is a data point saying the worst case did not occur.

Nothing about depth compounded either, and the reason is visible in the token counts. The operator was not summarising: its output grew with its input at a steady ≈ 7.5 tokens per term (490 tokens for 62 terms, 809 for 106, 1,540 for 203). The budget formula in `semantic_op` gives $\max(300, 1500(1 + 0.25d))$, so 2,625 tokens at depth 3, and the root fold used 1,540 of them — 59% headroom. Depth compounds loss for a lossy operator only when each level forces a re-encoding under pressure; here every level had room to copy, so the mechanism the module `docstring` warns about never engaged. That is a scale-conditional result, not a refutation, and Section 8 says where it stops holding.

7.3 Does the result actually differ from the exact one?

Yes, and the difference is measurable without leaving this run. Comparing against `real-tr-p8-full`’s stored glossary is tempting — 203 terms versus 206, with 16 keys unique to the semantic run and 19 unique to the exact one — but that comparison is worthless, because the eight term sheets themselves were produced by nondeterministic agent calls and differ between the two runs. The clean experiment is a within-run counterfactual: take this run’s own eight term sheets from the blob store, fold them with `agentmpi.ops.UNION`, and apply the harness’s deterministic tiebreak (`sorted(v)[0]` on each conflict list), which is precisely what `real-tr-p8-full` did to its own sheets.

That replay produces 203 keys — *the identical key set* — and 7 values that differ from the semantic result, 3.4% of the glossary. Exactly five terms attracted conflicting proposals from the population (*German*, *Irene Adler*, *brougham*, *cabman*, *reasoner*). The semantic operator resolved all five by choosing one of the two renderings that had actually been proposed and never invented a third, and it chose what `UNION`’s lexicographic tiebreak would have chosen on three of the five; *German* (country versus demonym) and *cabman* (“driver” versus “carriage driver”) are the two it decided differently. The remaining five differences are edits made to terms whose two inputs did not disagree at all: *Bohemian* and *European* were moved from bare place nouns to attributive forms, *mistress*

from “kept woman” to “lady of the house”, and — the interesting pair — *Miss Adler* and *Miss Irene Adler* were rewritten to use the transliteration of the surname the operator had already chosen for *Irene Adler*.

That pair is the whole argument for semantic reduction, in miniature and on real data. UNION resolved *Irene Adler* the same way the model did, but it treats *Miss Adler* as an unrelated key and left it spelling the surname the other way, because set union has no notion that two keys are about the same person. The semantic operator noticed and propagated. Seven agent calls and \$0.0894 therefore bought seven changed glossary entries: two cross-key consistency repairs an exact operator cannot express, two conflict resolutions decided against UNION’s arbitrary tiebreak, and three unforced substitutions whose merit is a matter of translation judgement rather than measurement.

The order-dependence the operator is warned about is also visible, in one instance. Table 1 counts six conflicts across the tree but the population only ever disagreed about five terms; the sixth is *Bohemian* at rank 0’s depth-2 fold, and it exists only because rank 2’s depth-1 fold had unilaterally rewritten that term one level earlier. A lossy operator does not merely resolve the disagreements it is given, it manufactures new ones for the levels above it, and under a different tree shape that particular conflict would never have arisen.

The harness’s own quality scoring cannot see any of it. Coverage, consistency, divergent entities, rendering verification, paragraph fidelity and length plausibility are identical between this run and **real-tr-p8-full** — 1.0, 1.0, 0, 1.0, 1.0, 1.0 — and identical again in **real-tr-p8-noglossary**, which skips the allreduce entirely. With six canonical shared entities across eight sections the metric is saturated, so no conclusion about translation quality can be drawn in either direction from this ablation as scored.

7.4 The cost, attributed

Against **real-tr-p8-full**: agent calls 31 versus 24, prompt tokens 48,530 versus 43,448, completion tokens 13,878 versus 8,636, cost \$0.3538 versus \$0.2599, messages 56 in both. Priced at the run’s own calibration (\$3/Mtok in, \$15/Mtok out), the seven merge calls alone account for \$0.0894 of the \$0.0939 difference, so cost attribution to the operator is essentially exact. Latency is where the naive comparison misleads. The reduce’s recorded `wall_s` went from 3.23s to 980.70s, a factor of 303, and the enclosing allreduce from 10.24s to 985.70s, a factor of 96. But rank 0’s 980.7s inside the reduce decomposes into 114.2s of its own model service, 507.7s of broker queue (3.6 + 499.3 + 4.8 across its three folds), and 358.6s spent waiting for rank 2’s depth-1 contribution, which was itself stuck behind rank 2’s 348.5s wait for a worker. So 866.3s — 88% — traces to worker attachment, directly or through a peer. The operator’s own contribution is 224.5s of model service across the seven calls, of which the critical path through the tree, $\max(\max(26.4, 23.7) + 39.0, \max(22.8, 21.2) + 34.8) + 56.6$, is 122.0s. Service time did grow with depth, roughly in proportion to output size: 23.5s mean at depth 1, 36.9s at depth 2, 56.6s at depth 3. A well-scheduled version of this run would therefore show a glossary phase of about two minutes rather than the sixteen recorded, against a genuinely free fold in the exact run.

7.5 The flagged findings

The **[warning]** on `halo_exchange` is a real defect in AgentMPI, and it is not specific to this configuration. The collective reports zero messages while the fabric logged 14 carrying its inter-

nal tag. Inspecting the recorded event shows the payload is `{label, left, right}` only: no `algorithm`, `size`, `rounds`, `messages_sent`, `tokens_sent` or `wall_s`. The same warning appears in `real-tr-p8-full`, `real-tr-p8-rdallreduce`, `real-tr-p4-full` and `real-tr-p2-full`, and is absent only from `real-tr-p8-nohalo`, which does not call the operation. It has since been diagnosed and fixed — commit `ee21d79` adds those fields to all three neighbourhood collectives and asserts on the trace rather than on the runtime counter — but the trace analysed here was recorded at commit `a6f6103`, before the fix, so the flag is accurate for this artefact and the derived `halo_exchange` row in the collective table (0 messages, 0 tokens, 0s, algorithm ?) should be read as missing data rather than as measurement.

The [\[note\]](#) on 15.2% parallel efficiency and 84.8% idle rank-seconds is real but misattributed if read as a property of semantic reduction. The reduction is genuinely serial — a $\log_2 p$ tree keeps at most four ranks busy at level 1 and one at level 3 — but the efficiency collapse is dominated by the 742.3s in which no rank at all was busy. The exact run with the identical harness scores 77.4%.

The [\[note\]](#) on a 258.7% coordination share is an artefact of the metric’s definition, explained above: per-rank collective walls are summed while ranks block concurrently. It should be read as “on average each rank spent 2.6 wall times’ worth of rank-seconds inside collectives”, which for eight ranks means roughly a third of the population’s time, not more than all of it.

The two [\[ok\]](#) findings are both correct and both worth keeping. Fifty-six reattachments, five to eight per rank and reaching incarnation 9 on ranks 0, 1, 4 and 6, is nearly double the 31 of `real-tr-p8-full`, and it is the same phenomenon as the queue stalls seen from the other side: each rank role outlived a succession of worker processes, and its mailbox, sequence numbers and context account survived every handover. Nothing was lost across an attachment boundary — 56 sends, 56 receives, zero failures, zero contract violations, zero retries. And all nine checkable collectives sent exactly the message count their closed forms predict.

7.6 Position in the ablation ladder

The harness docstring predicts that a semantic merge “would be none of those [exact, associative, commutative, idempotent], and the population would end up believing p slightly different glossaries”. The first clause remains a correct statement about the operator’s declared algebra; the second did not happen and, under `reduce_bcast`, cannot. The population divergence the docstring warns about is a property of the *algorithm*, not the operator: `recursive_doubling` has every rank perform its own fold sequence, which is why `algorithms.allreduce` passes `divergence_risk=op.lossy` on that path and a hardcoded `False` on the `reduce_bcast` path.

The sibling `real-tr-p8-rdallreduce` does not test that combination. Its configuration is `glossary_op: union` with `allreduce_alg: recursive_doubling`, so its operator is exact and its 16 `coll.allreduce` events all record `divergence_risk: false` — as they should, since recursive doubling with a canonical operand ordering over an exact operator gives every rank the same answer. No run in `traces/events/` sets `divergence_risk` true; the semantic-operator-plus-recursive-doubling cell of the design is empty. This run therefore supplies the cost of a semantic operator and the demonstration that `reduce_bcast` contains it, and says nothing at all about divergence. That cell is where the project’s divergence claim would have to be measured, and the two runs that exist do not measure it between them.

8 Threats to this reading

The wall-time comparison against `real-tr-p8-full` is not controlled. Both runs executed the same harness against real Cursor subagents, but this one lost 742.3s to worker attachment stalls and the other lost 2.3s. Nothing in the configuration explains the difference; the semantic operator does not spawn workers. Any sentence of the form “the semantic glossary made the run $6.9\times$ slower” is contaminated by an environmental variable neither run controlled, and the honest operator-attributable latency figure is the 122.0s critical path computed from service intervals. That figure in turn assumes queueing and service are cleanly separable in the broker log, which they are only if a claimed task begins executing immediately.

The zero-loss result is conditional on scale and on budget headroom, and this run cannot tell which. The root fold produced 1,540 tokens against a 2,625-token budget. The glossary size is set by the book, not by p — the $p = 1$, $p = 2$, $p = 4$ and $p = 8$ union runs report 208, 207, 208 and 206 terms — so at larger p the same ≈ 200 -term payload would pass through $\lceil \log_2 p \rceil$ folds with a budget that grows only as $1500(1 + 0.25\lceil \log_2 p \rceil)$. At $p = 8$ that is comfortable. It is also possible that this particular model would have copied faithfully at any budget and the headroom is irrelevant. One run at one p cannot separate those.

The seven-value difference is one sample of a nondeterministic operator. The whole point of `semantic_op` is that it is not reproducible, and this run folded the tree once. A second execution over the same eight term sheets would give a different set of edits, possibly a different count, and possibly a nonzero key loss. The 3.4% figure is a single draw, not an expectation, and the run contains no repeat from which to estimate variance.

“Better” is asserted, not measured. I judged two of the seven edits to be genuine cross-key consistency repairs and the rest to be plausible translation choices, by reading them. The harness’s quality scorer is saturated — identical values on every metric for the semantic, exact and no-glossary runs — so there is no instrument in this experiment that can distinguish a semantic merge from no merge at all on output quality. Any claim that the \$0.0894 bought something is currently a claim about the glossary, not about the book.

The counterfactual replay reuses code that has moved on. The UNION fold I ran to construct the comparison glossary is today’s `src/agentmpi/ops.py`, whereas the trace was recorded at commit `a6f6103`. UNION is a small and stable function and the `real-tr-p8-full` glossary it produced at run time is consistent with what today’s version produces, but the replay is a reconstruction rather than a recording, and the seven-value difference inherits that caveat.

Several derived metrics do not mean what they appear to. Agent latency p95 and maximum are dominated by broker queueing; the calibration constants α and β are fitted to those latencies; the coordination share exceeds 100% by summing concurrent blocking; the `halo_exchange` row is empty because of the instrumentation gap rather than because no traffic occurred; and `tokens_deferred` is reported as 8,893 in `metrics.json` but 40,799 in the `job.finish` event, the former being the tokens carried by the eight rendezvous-mode messages and the latter a wider count, so the same name denotes two quantities depending on where a reader looks. None of these invalidates the operator findings, which rest on event counts, token counts and blob contents rather than on derived timings, but each is a place where a reader could take a number from the headline table and be wrong about it.